

LABORATORIO P3 · PREDICCIONES · MPOO 2027-1

Te voy a preguntar 36 veces que crees que imprime una linea de codigo. Contesta con tu mejor intento y dale Enter. Si no sabes, adivina: fallar y entender por que es justo el ejercicio.

Escribe el valor como crees que se veria en pantalla. Por ejemplo:

3 3.5 true A null Infinity

BLOQUE 1 · Con que nace un atributo al que nadie le puso valor
Estos atributos se declararon, pero nunca se les asigno nada.

Asi estan declarados, fijate en el camelCase de cada nombre:

```
int    enteroSinValor;  
double decimalSinValor;  
boolean logicoSinValor;  
char    letraSinValor;  
String  textoSinValor;
```

Que imprime cada uno?

```
enteroSinValor  = 0  
decimalSinValor = 0.0  
logicoSinValor  = 0  
(int) letraSinValor = 0  
textoSinValor   = 0
```

RESULTADO DEL BLOQUE 1			
expresion	predijiste	salio	
enteroSinValor	0	0	ok
decimalSinValor	0.0	0.0	ok
logicoSinValor	0	false	FALLO
(int) letraSinValor	0	0	ok
textoSinValor	0	null	FALLO

Fallaste 2. Anotalas: explicarlas es lo que se califica.

logicoSinValor. Porque confundí el valor por defecto de un tipo primitivo boolean (que es false) con el de un tipo numérico como int (que es 0). En Java, las variables de tipo boolean no se inicializan a 0, sino a false.

textoSinValor. Porque String es un tipo objeto (referencia), no un tipo primitivo. El valor por defecto para cualquier tipo objeto en Java es null, no 0 (que es para numéricos).

```
=====
BLOQUE 2 · La division que miente
Fijate en el tipo de cada operando y en donde esta puesto el casting.
=====
```

Que imprime cada expresion?

```
7 / 2 = 3
7.0 / 2 = 3.5
(double) 7 / 2 = 3.5
(double) (7 / 2) = 3
7.0 / 0 = 0.0
0.0 / 0.0 = 0.0
```

RESULTADO DEL BLOQUE 2				
expresion	predijiste	salio		
7 / 2	3	3	ok	
7.0 / 2	3.5	3.5	ok	
(double) 7 / 2	3.5	3.5	ok	
(double) (7 / 2)	3	3.0	ok	
7.0 / 0	0.0	Infinity	FALLO	
0.0 / 0.0	0.0	NaN	FALLO	

Fallaste 2. Anotalas: explicarlas es lo que se califica.

Infinity. Pensé que dividir entre cero en punto flotante daría 0.0 por algún tipo de protección, pero en realidad la especificación IEEE 754 define que dividir un número positivo entre 0.0 da Infinity (infinito positivo).

NaN. Asumí que 0 dividido entre 0 daría 0, pero en aritmética de punto flotante, 0.0/0.0 es una operación indefinida, por lo que el resultado es NaN (Not a Number).

```
=====
BLOQUE 3 · El modulo, y para que sirve de verdad
El % da el residuo: sirve para saber si algo es par y para sacar digitos.
=====
```

Que imprime cada expresion?

```
7 % 2 = 1
10 % 5 = 0
-7 % 2 = -3
7 % 2.5 = 2.0
12345 % 10 = 5
12345 / 10 = 1234
```

RESULTADO DEL BLOQUE 3				
expresion	predijiste	salio		
7 % 2	1	1	ok	
10 % 5	0	0	ok	
-7 % 2	-3	-1	FALLO	
7 % 2.5	2.0	2.0	ok	
12345 % 10	1	5	FALLO	
12345 / 10	2	1234	FALLO	

Fallaste 3. Anotalas: explicarlas es lo que se califica.

-1. Confundí el operador módulo (%) con la división entera (/). El módulo devuelve el resto de la división, no el cociente. En este caso, -7 dividido entre 2 da cociente -3 y resto -1 (porque $-7 = 2 * (-3) + (-1)$).

5. Calculé mal mentalmente. El módulo % 10 devuelve la última cifra del número. 12345 termina en 5, no en 1. El 1 es la primera cifra, no la última.

234. Confundí la operación. $12345 / 10$ es división entera, que trunca el decimal y da 1234 (no 2, que sería $12345 \% 10$ o confundí con el número de cifras). El 2 no tiene relación con este cálculo.

```
=====
BLOQUE 4 · El casting y lo que se pierde
Un casting no redondea: corta. Y un tipo chico no aguanta cualquier numero.
=====
Que imprime cada expresion?
(int) 3.9 = 3
(int) -3.9 = -3
(char) 65 = A
(int) 'A' = 65
'A' + 1 = 66
(char) ('A' + 1) = B
(byte) 200 = -56

+-----+-----+-----+-----+
| RESULTADO DEL BLOQUE 4 |
+-----+-----+-----+-----+
| expresion | predijiste | salio | |
+-----+-----+-----+-----+
| (int) 3.9 | 3 | 3 | ok |
| (int) -3.9 | -3 | -3 | ok |
| (char) 65 | A | A | ok |
| (int) 'A' | 65 | 65 | ok |
| 'A' + 1 | 66 | 66 | ok |
| (char) ('A' + 1) | B | B | ok |
| (byte) 200 | -56 | -56 | ok |
+-----+-----+-----+-----+
Sin fallos en este bloque.
```

```
=====
BLOQUE 5 · Relacionales y logicos
El && se detiene en cuanto ya sabe la respuesta: eso es el corto circuito.
=====
Aquí vidasRestantes vale 0. Que imprime cada expresion?
5 > 3 = true
5 == 5.0 = true
0.1 + 0.2 == 0.3 = true
true && false = false
true || false = null
vidasRestantes > 0 && 10 / vidasRestantes > 1 = false

+-----+-----+-----+-----+
| RESULTADO DEL BLOQUE 5                                     |
+-----+-----+-----+-----+
| expresion          | predijiste   | salio        |          |
+-----+-----+-----+-----+
| 5 > 3              | true         | true         | ok       | | |
| 5 == 5.0           | true         | true         | ok       |
| 0.1 + 0.2 == 0.3   | true         | false        | FALLO   |
| true && false       | false        | false        | ok       |
| true || false      | null         | true         | FALLO   |
| vidasRestantes > 0 && 10 / vi~ | false        | false        | ok       |
+-----+-----+-----+-----+
Fallaste 2. Anotalas: explicarlas es lo que se califica.
```

false. Porque los números decimales en binario (como 0.1 y 0.2) no se representan de forma exacta. Al sumarlos, el resultado es ligeramente mayor que 0.3 (0.30000000000000004), por lo que la comparación da false.

true. No tiene sentido que diera null, ya que || es un operador lógico que siempre devuelve un boolean. Confundí con otro contexto o simplemente fue un error conceptual: true || false siempre es true.

```
+-----+-----+-----+-----+
| RESULTADO DEL BLOQUE 6                                     |
+-----+-----+-----+-----+
| expresion          | predijiste   | salio        |          |
+-----+-----+-----+-----+
| 0.1 + 0.2          | 0.3          | 0.300000~   | FALLO   |
| Integer.MAX_VALUE + 1 | -2147483648 | -2147483~   | ok       |
| 5 + 3              | 8            | 8            | ok       |
| "5" + 3            | 53           | 53           | ok       |
| 1 + 2 + "3"        | 33           | 33           | ok       |
| "1" + 2 + 3        | 15           | 123          | FALLO   |
+-----+-----+-----+-----+
Fallaste 2. Anotalas: explicarlas es lo que se califica.
```

0.30000000000000004. Por el mismo problema de precisión de punto flotante. 0.1 y 0.2 no son exactos en binario, y al sumarlos el resultado tiene un pequeño error de redondeo.

123. Porque el operador + en Java es asociativo por izquierda. Primero se evalúa "1" + 2, que al ser String + int, convierte el 2 a String y concatena → "12". Luego "12" + 3 → "123". Pensé que sumaría los números primero, pero el String al inicio fuerza la concatenación.

```
#####
# TU TABLA FINAL · ESTA ES LA CAPTURA QUE TIENES QUE ENTREGAR #
#####
# expresion predijiste salio
-----
1 enteroSinValor 0 0 ok
2 decimalSinValor 0.0 0.0 ok
3 logicoSinValor 0 false FALLO
4 (int) letraSinValor 0 0 ok
5 textoSinValor 0 null FALLO
6 7 / 2 3 3 ok
7 7.0 / 2 3.5 3.5 ok
8 (double) 7 / 2 3.5 3.5 ok
9 (double) (7 / 2) 3 3.0 ok
10 7.0 / 0 0.0 Infinity FALLO
11 0.0 / 0.0 0.0 NaN FALLO
12 7 % 2 1 1 ok
13 10 % 5 0 0 ok
14 -7 % 2 -3 -1 FALLO
15 7 % 2.5 2.0 2.0 ok
16 12345 % 10 1 5 FALLO
17 12345 / 10 2 1234 FALLO
18 (int) 3.9 3 3 ok
19 (int) -3.9 -3 -3 ok
20 (char) 65 A A ok
21 (int) 'A' 65 65 ok
22 'A' + 1 66 66 ok
23 (char) ('A' + 1) B B ok
24 (byte) 200 -56 -56 ok
25 5 > 3 true true ok
26 5 == 5.0 true true ok
27 0.1 + 0.2 == 0.3 true false FALLO
28 true && false false false ok
29 true || false null true FALLO
30 vidasRestantes > 0 && 10 / vida~ false false ok
31 0.1 + 0.2 0.3 0.30000000~ FALLO
32 Integer.MAX_VALUE + 1 -2147483648 -2147483648 ok
33 5 + 3 8 8 ok
34 "5" + 3 53 53 ok
35 1 + 2 + "3" 33 33 ok
36 "1" + 2 + 3 15 123 FALLO
-----
ACERTASTE 25 DE 36
```

```
BLOQUE 7
7.1 = 2.5
7.2 = true
7.3 = 9
7.4 = true
7.5 = false
(si todavia no escribes nada aqui, no se imprime nada: es normal)
```